Comp 4915

Special Topics in MIS

- UNIX
- Windows 2016 Server

Helpful Reference

- The Linux Command Line, 2nd Edition: A Complete Introduction
- Very easy read that starts at the basics and focusses on the command line
- Supplements the text which is dense and slow reading
- Covers bash, the file system, basic utilities such as vim and sed

Instructor

- Bruce Link
- Office: SW2-365
- BCIT email: bruce_link@bcit.ca
- BCIT phone: (604) 412-7508
- BCIT fax: (604) 434-4496

This Week's Topics

- Outline and course review
- Welcome to UNIX, Chapter 1
- Introduction to Fedora and Red Hat Enterprise Linux, Chapter 4
- The Shell, Chapter 5

Course Goals

- Two key technologies in IS
- UNIX operating system
  - work comfortably on a UNIX platform
  - install and configure UNIX systems
  - create simple scripts.
- Windows 2016 Operating System
  - learn components and architecture of NT
  - install and configure NT

UNIX Subgoals 1/2

- Explain how UNIX came into existence, and its relevance in the workplace
  - Explain the Systems Administrator tasks
  - Manage Linux file systems
  - Program the UNIX shell
  - Configure networking and file sharing

UNIX Subgoals, 2/2

- Work comfortably in the bash-shell
- Use UNIX utilities such as awk, grep, sed, wc, or diff, perl
- Set up internet services
- Set up firewalls

Chapter 1: Welcome to Linux

- GNU-Linux Connection
- UNIX History
- Why Linux?
- Linux Overview
- Additional Features of Linux

GNU-Linux Connection

- 1983: Richard Stallman announces GNU project for creating operating system
  - GNU: GNU Is not UNIX
- GNU based on freedom of use philosophy
- 1991: GNU project had most of OS except kernel
- GNU software runs on several kernels
  - Linux, FreeBSD, NetBSD, Hurd, Darwin

Characteristics of UNIX

“UNIX is simple, but it takes a genius to understand the simplicity”

- *Dennis Ritchie, a c/unix creater*

What is UNIX?

- Multitasking Operating System
- Multi-user
- Memory protection between processes
- Written in C
- Available on many hardware platforms
- Wide use in academic, engineering, internet application areas

History of UNIX

- 1969 - Thompson and Ritchie of AT&T write UNIX as a simple version of MULTICS
- 1970 - C language designed by Ritchie
- 1973 - UNIX rewritten in C
- 1974 - AT&T licenses UNIX to universities
- 1979 Great schism : BSD and AT&T System V
  - BSD: SunOS, Ultrix
  - System V: HP-UX, SGI IRIX, SCO UNIX, IBM AIX

History of UNIX

- 1984 MIT develops X Window which becomes the standard UNIX windowing system
- 1988 System V Release 4 merges BSD and System V
- 1991 OSF/1 released (everyone else)
- POSIX standard defines a standard set of system calls and utilities
  - Which everyone then ignores

History of UNIX: Linux

- 1991 - Linus Torvalds starts building Linux
  - Independent implementation of UNIX-like kernel
  - Good time frame as GNU software available
- 1994 - release 1.0 kernel
- 1996 - release 2.0 kernel
- 2003 – release 2.6 kernel
- Current stable kernel 6.18.4
  - See http://www.kernel.org/

Linux Operating System

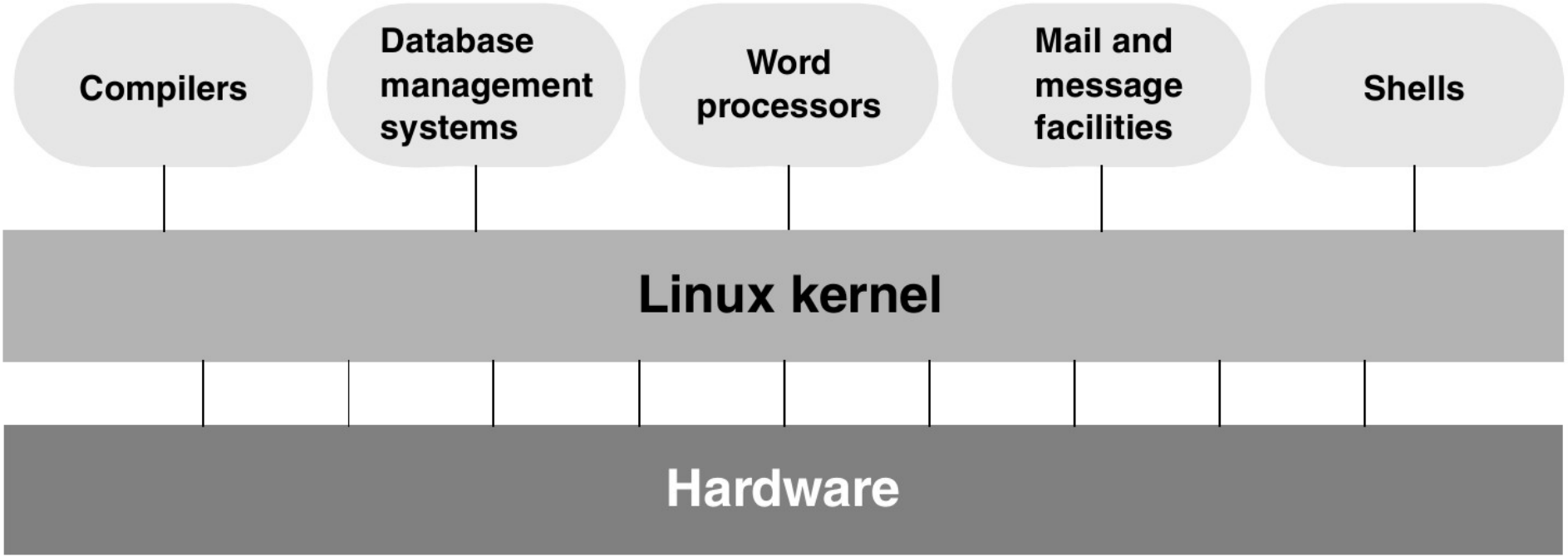

Figure 1-1 A layered view of the Linux operating system

What is So Good About Linux?

- UNIX much more efficient than windows OS
  - Dominant on server side, created internet
- Linux is open source UNIX-like OS
  - Wide variety of applications
  - Good support for peripherals
  - Good support for developers
  - Massive friendly community of contributors
  - Good support from IBM, Sun, most non-MS companies

Overview of Linux

- Multiuser, each protected from others
- Protected multitasking operating system
- Secure hierarchical file system
- “Shell” - a command interpreter and programming language
- Hundreds of useful utilities
- Choice of windows managers
- A “real” operating system

The Shell: Command Interpreter and Programming Language

- Windows command line is a shell of a real shell!
  - The UNIX command interpreter is called a shell
  - Several available, most popular: bash
- Includes script language, filename generation, device-independent I/O, job control, inter-process communication
- Many shell functions, useful utilities
- All system administration facilities available
- Essential to know

Additional Features of Linux

- Most features of UNIX versions (BSD, System V, Solaris) present in Linux
  - Why did Apple go with FreeBSD instead for Mac?
- Recent emphasis has been improving GUI experience
  - Built using window manager on X Window System
  - GNOME and KDE desktop managers most popular
- Services for running networks all present
- Predominant software development platform

Chapter 4: Getting Started with Fedora / Red Hat Enterprise Linux

- Controlling superuser/root access
- Tour of the Fedora/RHEL Desktop
- Where to find documentation
- More about logging in
- Controlling Windows

Getting Started

- Critical part of course is learned in the labs
- Linux more command-oriented than drag-and-drop
  - Need to practice so you fingers learn
  - Become fluent with the command-line commands
  - Know command-line equivalent even though a GUI version is available
- Choose ether GNOME or KDE user interface
  - Both should be installed

Learning Systems Administration Required Knowledge

- The Linux operating system comes in over a dozen different distributions
  - Not every distributions has same directory structure or starts in the same way
- We talk here about common principles and how things are often done
- With a specific version, you may have to hunt things out to find specifics
  - E.g. switch from inetd to xinetd

System Administrator

- System administrator responsible for keeping system running and users happy
- To do this requires greater knowledge about how things work than windows
- If you are running your own system, you are the system administrator
  - On a large system, it’s a full time job

Systems Administration and Superuser

- System administrator requires powers not required by normal users
  - Halting the system
  - Changing file permissions
  - Creating or removing any file in any directory
  - Changing user's passwords
- Special user account root has these powers
  - Can set up other users to be able to exercise some of these in a controlled way

Exercising Superuser Privileges

- Running in single user mode
- Logging in as root in multiuser mode
  - Don't do this
- Executing su to start a shell as superuser
- Running a command which has suid mode to become superuser
- Running the sudo command
  - Allows users to execute selected superuser commands (which are logged)

Where are System Administration Tools?

- Most commands kept in /sbin and /usr/sbin
- "s" stands for statically linked
  - Do not use dynamic libraries
  - Can be used even when there are problems with /usr/lib
- Probably want these in your path
  - PATH=/sbin:/usr/sbin:$PATH
- Otherwise use full pathname, e. g. /sbin/ifconfig

Tour of the Fedora/RHEL Desktop

- Desktop managers evolve, so details will vary depending on version you are running
  - Text based on Fedora 19, RHEL 7
    - new version of Fedora comes out every 6 months or so
  - Fedora aggressive about updating older versions
    - Older versions will get improvements, but count on things changing
- Run through this text's section in lab and notice any differences

KDE 4 Getting Started

- Tutorials on KDE 4
  - Usage: http://userbase.kde.org/
  - Development: http://techbase.kde.org/Development/Tutorials
- To add application to panel (lower bar)
  - Locate application in application launcher, right click on it and select “add to panel”
- To change application launcher to be as in text
  - Right click on it and select classical menu style

Where to Find Documentation

- KDE Help center (under Applications:Help)
  - Some is still based on version 3
  - Read entry under Plasma
- GNOME Help (under System:Help in top panel)
- FAQs and HOWTOs
  - Linux Documentation Project, http://tldp.org/
- man pages and info/pinfo
- Online documentation in /usr/share/doc

Use man to Display System Manual

- Displays the system manual from terminal
- E.g. man ls
  - Displays the manual on the command ls
- Try man man to see the definition for the man command
- Try man bash to see the (long) short documentation on the shell
  - Command line interpreter

Use info/pinfo to Display System Information

- The info utility has a tutorial on itself and documents many of the linux programs
- Highly keyboard-key oriented
  - C-h means CONTROL-h (at same time)
  - M-h means ALT-h (at same time)
- Run through tutorial in lab
- Info generally gives in-depth documentation of selected tools
- pinfo is alternative (may be easier to use)

More About Logging In

- Have a regular user account with a password!
  - Never use the root account to login
- On graphical login screen can select language and which desktop to use
- Virtual text terminals allow login
  - C+A+F*n* switches to n^{th} terminal (5 or 6 by default)
  - One of them is graphics terminal (F1 on mine)
- Login remotely using `ssh` or `telnet`

Terminal Sessions

- Bringing up a terminal window or logging into a terminal have similar properties
  - Shell prompt indicates system waiting for command to be entered
  - Shell prompt can be configured, default will be your username, host, and working directory
    - Followed by $ for normal user, # for root
- If system boots into terminal, can start GUI with `startx` command

Command Editing

- Default character to erase character before cursor is backspace key
  - Delete key should erase character under cursor
- Ctrl-W deletes word to left of cursor
  - Delimited by spaces and/or tabs
- Ctrl-U deletes to beginning of line
- Can move cursor using arrow keys
- Ctrl-Z interrupts program, Ctrl-C usually kills it

Controlling Windows

- With multiple open windows input focus determines active window
  - receives keyboard input
- Way of changing input focus is configurable
  - Click-to-focus, focus-follows-mouse, focus-strictly-under-mouse
  - KDE: under System Setting: Window Behavior
  - GNOME: under System: Preferences: Look and Feel: Window Preferences

Window Manager

- Desktop manager (e.g. KDE, GNOME) control how applications are started, and relate to each other
- Window manager controls look and feel of basic GUI
  - How windows look and are manipulated
- Many choices available
- For fancier 3D effects, enable desktop effects

Chapter 5: The Shell

- What a shell is
- Basic features of shells
- How shell processes command
- How redirection works
- How filename expansion works
- Later unit will cover the bash shell in more detail

Shells

- Shell: user program that acts as a command line interpreter
- Shell is distinct from the kernel
- User can choose which shell to use
- Common unix shells:
  - Bourne shell (sh), C shell (csh) & Korn shell (ksh)
- Common linux shells:
  - Bourne-again shell (bash), TC shell (tcsh), Z shell (zsh)
- Sets environment for other processes (process inherits its parent’s environment)

Functions of the Shell

- Searches for command & executes associated program
- Substitutes shell variable values
- Performs command substitution
- Performs filename generation
- Completes filenames
- Handles I/O redirection & pipelines
- Provides interpreted programming interface

Command Line Syntax

- *Command [arg1] [arg2] … [argn]* ENTER
  - arguments separated by spaces (tabs too)
  - argument is something command acts upon
  - option is an argument that modifies what command does
  - many commands start options with - or --
  - sometimes can combine options (see man pages for each command)
  - ex: ls -r -x is the same as ls -rx

Special Characters In Shells

- Characters special to the Bash shell:
  - ~ ` # $ & * ( ) \ | [ ] { } ; ' " < > / ? !
- If you don't want these to be treated specially, enclose all arguments containing these in single quotes '
- Can precede single character with backslash to escape its meaning \
- Escaping the RETURN key gives continuation lines

Control Characters

- Often interpreted by tty driver or shell
- Use stty command to find out what control keys used by driver (and change them)
- Common Keys at tty driver level
  - Ctrl-C Stop current command
  - Ctrl-D End of input
  - Ctrl-\ Stop current command (try Ctrl-C first)
  - Ctrl-S Stop output to screen
  - Ctrl-Q Restart output to screen
  - DEL or Ctrl-? Erase last character
  - Ctrl-U Erase entire line
  - Ctrl-Z Suspend current command

Bash Command Line Editing

- Arrows can be used to move back and forth in the current or previous commands
- BACKSPACE deletes one character backward
- DEL or Ctrl-D deletes one character forward
- Ctrl-R searches backwards in the command history file
- TAB attempts textual completion of names
  - Commands, files, directories
  - Multiple possibilities will beep
- Ctrl-V inserts the next character as is

Command Processing

- Shell takes line and breaks it into *words* or *tokens*
  - separated by spaces
- First token taken as command name
- If command is a relative pathname, shell looks through directories in *path* for the command file
- If command file is found, shell starts a new process to run the command with the arguments
- Shell sleeps until the command is finished
- Then shell prompts for the next command

Standard Input and Standard Output

- Most commands are written using the special files
  - *standard input, standard output, and standard error*.
- Commands read from *standard input*
  - Defaults to the terminal
- Commands write normal output to *standard output*
  - Defaults to the terminal
- Commands may write error messages to *standard error*
  - Defaults to the terminal

Redirection

- Standard I/O files can be redirected within the shell as follows:

  `>` redirect `stdout` to file

  `>>` redirect and append `stdout` to file

  `<` redirect `stdin` from file

  `2>` redirect `stderr` to file

  `&>` redirect `stdout` and `stderr` to file

  Some examples:

```
$ cat
$ cat first last > first+last
$ cat < first+last
$ cat < last >> first
$ cat < last > first
```

Pipes

- A pipe connects the stdout of one process to the stdin of another process

  Examples:

```
$ cat food | more
$ man du | col -b | lpr
$ du -s /home/unix/* | sort -nr | head
```

- The tee utility allows us to send the output of one process to both stdout and to a file

  Example:

```
$ sort garbage | tee trash | head
```

Background Command Execution

- Append `&` to start a command in the background
  - eg: `find / -print > foo &`
- Can put the foreground job in the background by:
  - Stopping it with **`^Z`**
  - Typing **`bg`** (sends most recently stopped job to background, or **`fg`** to resume the command in the foreground)
- Can see the background jobs with **`jobs`**

```
$ jobs
[1] - Stopped      man batch
[2] - Stopped      find /usr -print
[3] + Stopped      man jobs
```

- To move a job to the foreground type `fg %<jobnum>`

```
$ fg %2
```

Filename Generation

- Following characters can be used to match existing filenames:

`*`	matches any number of characters
?	matches any single character
`[list]`	matches any character in list, can include range a-b
`[!list]`	matches any character not in list

- Can also use wildcards in pathnames
  - E.g: `$ ls /us*/[be1-3]*`

Commands to Know

- tr
- tee
- bg
- fg
- jobs

Appendix A: Regular Expressions

- Many times want to match a variety of things
  - E.g. all names beginning with B or b
- UNIX (and other contexts) often use *regular expressions* to match string patterns
- Different contexts can use different syntax
  - File name and string search syntax differ
- Cover string pattern version here
  - Used by ed, sed, vim, emacs, grep, gawk, etc.
  - File name version later

Simple Regular Expressions 2

- `*` matches 0 or more occurrences of the character matched to its left
  - `/ab*c/` matches ac, abc, abbc, abbbc
  - `/ab.*c/` matches abc, abxc, ab45c, ab 34.332x c
  - `/[a-zA-Z ]*/`matches a string of letters and spaces
  - `/(.*)/` matches as long a string as possible between `(` and `)`
- A regular expression matches the longest possible string, starting as far left as possible

Simple Regular Expressions 3

- Regular expression beginning with ^ must start at the beginning of a line
- Regular expression ending with $ must end at the end of a line
  - /^T/ matches a T at the beginning of a line
  - /:$/ matches a colon that ends a line
- Special character can be preceded by \ to represent itself (“quoting it”)
  - /end\./ matches end.

Simple Regular Expressions 4

- Some utilities allow bracketing expressions with quoted parentheses, \(and \)
  - The quoted parentheses are not matched but just select a part of the matched string
- Editors which have search and replace may have special regular expressions for the replacement part
  - Ampersand, &, represents the matched string
  - Quoted digits, \1, \2, ..., *n*, match the nth bracked expression

Summary

- We have gone from an overview of UNIX/Linux to start our coverage of the bash shell
- This will be used frequently and it important to become comfortable using it
- Next week will cover the file system and basic command line utilities